



**DAYANANDA SAGAR
UNIVERSITY**

TRAINING PROJECT REPORT ON

SHOPPING CART SYSTEM

SUBMITTED BY:

- 1. SOWMYA .**
- 2. SADIA .**
- 3. SHUBHSANKET .**
- 4. KEDARNATH .**
- 5. VINEETH .**

Course: MCA

Year: 2026

INDEX

1.INTRODUCTION

2.Objective of the Project

3.Scope of the Project

4.Feasibility Study

5. Existing System

6. Proposed System

7. System Design

8. Entity–Relationship Diagram

9. Database Design

10. Coding

11.SYSTEM IMPLEMENTATION

12.WORKING LOGIC

13.OUTPUT

14. Conclusion

15. Future Enhancement

Introduction

The Shopping Cart System is a console-based application developed using Core Java with JDBC (Java Database Connectivity) and MySQL as the backend database. This project is designed to simulate the basic functionality of an online shopping cart, where users can manage products by adding items, removing items, viewing selected items, and calculating the total bill.

In today's digital world, shopping cart systems are an essential part of e-commerce platforms. This project provides a simplified implementation of such a system, focusing on the fundamental operations and database interactions. The application uses JDBC to establish a connection between the Java program and the MySQL database, enabling persistent storage and retrieval of cart data.

The main objective of this project is to demonstrate how Java applications can interact with relational databases using JDBC. It also helps in understanding core programming concepts such as user input handling, control structures, and database operations like insertion, deletion, and retrieval.

This system is designed as a console-based application to keep the implementation simple and focused on backend logic rather than graphical interfaces. Despite its simplicity, it effectively illustrates real-world concepts used in larger e-commerce applications.

This project emphasizes the importance of data persistence by storing cart details in a database rather than temporary memory. It also demonstrates the use of SQL queries for efficient data manipulation and retrieval. The application is modular in design, making it easy to understand and extend with additional features. Furthermore, this project serves as a foundation for developing more advanced web-based or GUI-based shopping systems.

Overall, it enhances practical knowledge of integrating Java applications with database systems

Objectives of the Project

The primary objective of the **Shopping Cart System** is to design and develop a console-based application that demonstrates the integration of Core Java with a relational database using JDBC. This project aims to provide a clear understanding of how Java applications can interact with databases to perform real-time data operations efficiently.

One of the key objectives is to implement basic shopping cart functionalities such as adding items, removing items, viewing the cart contents, and calculating the total bill. These features help simulate a real-world shopping experience in a simplified manner. The project also focuses on performing CRUD operations (Create, Read, Update, Delete) using SQL queries, thereby strengthening the understanding of database management concepts.

Another important objective is to gain practical knowledge of JDBC, including establishing database connections, executing SQL statements using Statement and PreparedStatement, and handling data using ResultSet. It also aims to highlight the importance of using PreparedStatement for improved performance and security against SQL injection.

The project is designed to enhance programming skills in Java, especially in areas such as user input handling, control structures, exception handling, and modular coding practices. By working on this system, developers can understand how to structure a real-

time application and manage interactions between different components effectively.

Additionally, this project aims to build a strong foundation for developing more advanced applications, such as web-based e-commerce systems or GUI-based shopping platforms. It encourages logical thinking, problem-solving abilities, and practical implementation of theoretical concepts learned in database management and Java programming.

Finally, the project serves as an academic exercise to help students prepare for real-world software development scenarios, where database connectivity and backend logic play a crucial role in application design and performance.

Scope of the Project

The scope of the Shopping Cart System is focused on developing a console-based application that performs essential shopping cart operations using Core Java and JDBC with a MySQL database. This project is limited to implementing the backend logic and database connectivity required to manage cart items efficiently.

The system allows users to perform basic operations such as adding items to the cart, removing items, viewing the list of selected products, and calculating the total bill. All data is stored in a MySQL database, ensuring persistence and enabling users to manage their cart data reliably during program execution.

The project is designed for educational purposes to demonstrate how Java applications interact with databases using JDBC. It covers fundamental concepts such as establishing database connections, executing SQL queries, handling user input, and managing

exceptions. The scope does not include advanced features such as graphical user interfaces, online payment integration, user authentication, or multi-user support.

Although the application is simple, it can be extended in the future to include additional functionalities such as updating item quantities, applying discounts, generating invoices, and integrating with web or mobile interfaces. The current implementation provides a strong foundation for understanding real-world application development and can be enhanced further based on requirements.

Overall, the project scope is confined to building a functional, console-based shopping cart system that emphasizes core programming, database interaction, and logical problem-solving skills.

Feasibility Study

The feasibility study is an important step in system development that evaluates whether the proposed Shopping Cart System is practical, achievable, and beneficial. It helps determine if the project can be successfully developed within the given constraints such as time, cost, and available resources. The feasibility of this project can be analyzed under the following aspects:

1. Technical Feasibility

The project is technically feasible as it uses widely available and well-supported technologies such as Core Java, JDBC, and MySQL. These technologies are easy to learn, implement, and integrate. The system does not require any advanced hardware or complex software tools. A basic computer system with Java Development Kit

(JDK), MySQL Server, and an IDE (such as Eclipse or IntelliJ) is sufficient to develop and run the application. Therefore, the technical requirements are minimal and easily manageable.

2. Economic Feasibility

The project is economically feasible as it involves no significant financial investment. All the tools and technologies used are either open-source or freely available. There is no requirement for expensive licenses, hardware upgrades, or external services. Since the system is a small-scale academic project, the development and implementation cost is very low, making it cost-effective.

3. Operational Feasibility

The system is user-friendly and operates through a simple console interface, making it easy for users to understand and use. Basic input and output operations are used to interact with the system. The features provided, such as adding items, removing items, and calculating the bill, are straightforward and require minimal training. Hence, the system is operationally feasible and can be used without difficulty.

4. Time Feasibility

The project can be completed within a short period, making it suitable for academic purposes. Since the functionalities are limited and clearly defined, development, testing, and implementation can be done within the allocated time frame. This ensures that the project is feasible in terms of time constraints.

5. Legal Feasibility

The project does not violate any legal or regulatory requirements. It uses standard programming tools and does not involve handling sensitive personal data or copyrighted material. Therefore, it is

legally safe to develop and use this applicatio

Existing System

In the current scenario, many shopping cart systems are part of large-scale e-commerce platforms such as web or mobile applications. These systems are designed with advanced features like graphical user interfaces, user authentication, payment gateways, product recommendations, and real-time inventory management. While these systems are highly efficient and user-friendly, they are also complex and require advanced technologies and frameworks for development and maintenance.

Traditional or existing systems used for learning purposes are often limited to manual methods or basic applications where data is not stored permanently. In some cases, shopping cart functionalities are implemented using simple programs where data is stored temporarily in memory. This means that once the program is closed, all the data is lost, making it inefficient for real-world use.

Moreover, many existing academic-level systems do not emphasize proper database connectivity. They lack the use of structured database management systems, resulting in poor data handling and limited scalability. These systems may also not demonstrate real-time interaction between applications and databases, which is an important concept in modern software development.

Another limitation of existing simple systems is the absence of modular design and proper error handling. This makes them difficult to maintain and extend with additional features. They often do not follow best practices such as using prepared statements for secure database operations, which can lead to vulnerabilities like SQL injection.

Proposed System

The proposed Shopping Cart System is a console-based application developed using Core Java, JDBC, and MySQL to provide a simple and efficient way of managing shopping cart operations. This system is designed to overcome the limitations of existing basic systems by introducing database connectivity and persistent data storage.

In the proposed system, users can perform essential operations such as adding items to the cart, removing items, viewing all items in the cart, and calculating the total bill. Unlike traditional basic programs where data is stored temporarily, this system stores all cart details in a MySQL database, ensuring that the data is maintained consistently during the application runtime.

The system uses JDBC (Java Database Connectivity) to establish a connection between the Java application and the MySQL database. Through this connection, SQL queries are executed to insert, delete, retrieve, and manage data efficiently. The use of PreparedStatement ensures secure and optimized database operations by preventing SQL injection and improving performance.

The application is designed with a modular structure, separating database connectivity and business logic, which makes the system easy to understand, maintain, and extend. It also includes basic error handling mechanisms to ensure smooth execution and handle invalid inputs effectively.

Although the current system operates through a console interface, it provides a strong foundation for future enhancements. It can be extended into a graphical user interface (GUI) application or integrated into web-based platforms with additional features such

as user authentication, product catalogs, payment processing, and report generation

7. System Design

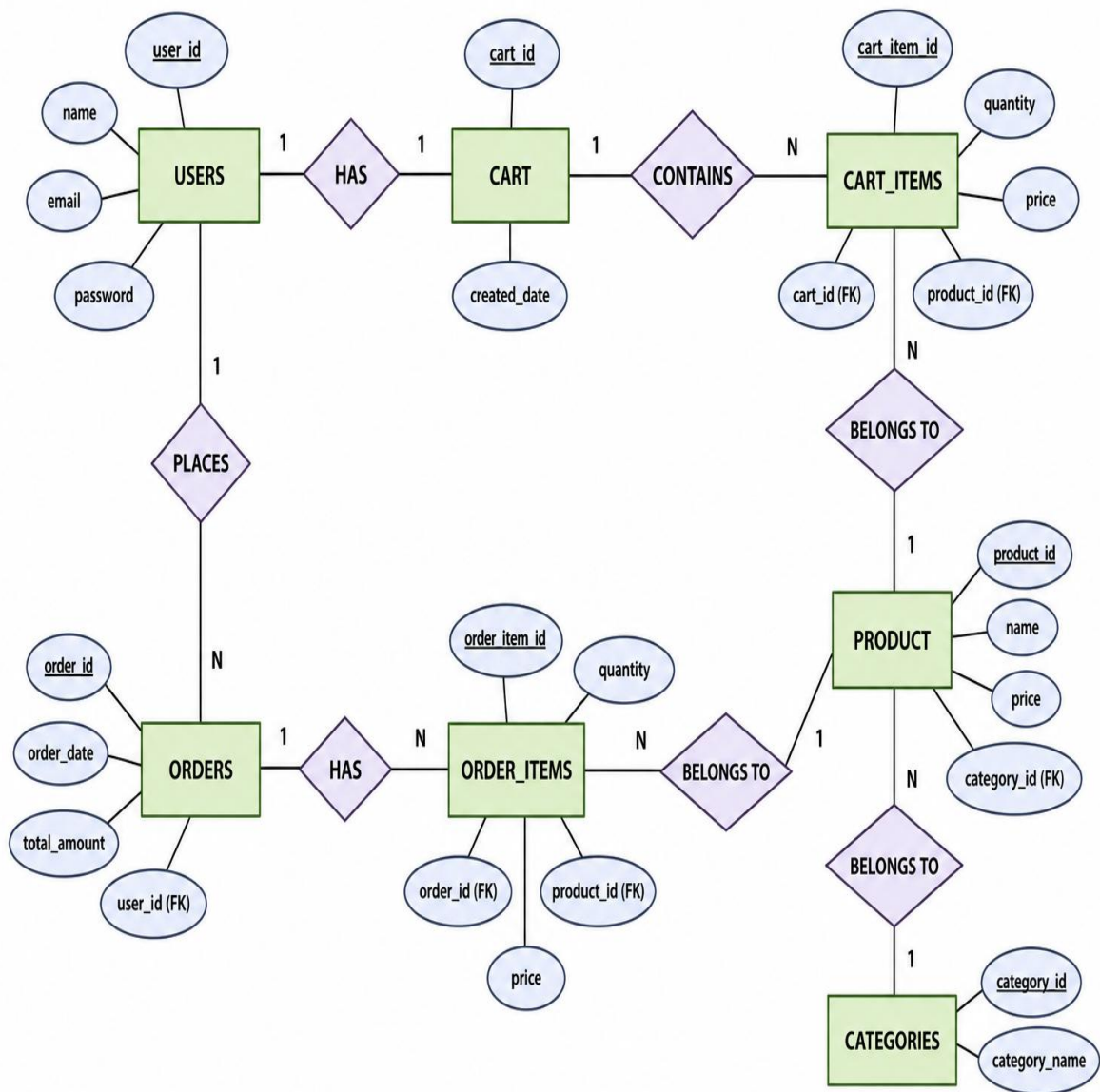
Architecture:

- Presentation Layer → Console Interface
- Business Logic Layer → Java Methods
- Data Access Layer → JDBC
- Database Layer → MySQL

Modules:

- Add Item
- Remove Item
- View Cart
- Generate Bill
- Order Management

ER-DIAGRAM



9. DATABASE DESIGN

```
drop DATABASE shopping_cart;
```

```
CREATE DATABASE shopping_cart;
```

```
USE shopping_cart;
```

```
-- USERS
```

```
CREATE TABLE users (
```

```
    user_id INT PRIMARY KEY AUTO_INCREMENT,
```

```
    username VARCHAR(50),
```

```
    password VARCHAR(50),
```

```
    mobile VARCHAR(15)
```

```
);
```

```
-- ADMIN
```

```
CREATE TABLE admin (
```

```
    admin_id INT PRIMARY KEY AUTO_INCREMENT,
```

```
    username VARCHAR(50),
```

```
    password VARCHAR(50)
```

```
);
```

```
-- PRODUCTS
```

```
CREATE TABLE products (  
    product_id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(100),  
    price DOUBLE  
);
```

```
-- CART
```

```
CREATE TABLE cart (  
    cart_id INT PRIMARY KEY AUTO_INCREMENT,  
    user_id INT,  
    product_id INT,  
    quantity INT  
);
```

```
-- ORDERS
```

```
CREATE TABLE orders (  
    order_id INT PRIMARY KEY AUTO_INCREMENT,  
    user_id INT,
```

```
total_amount DOUBLE,  
order_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

-- ORDER ITEMS

```
CREATE TABLE order_items (  
    order_item_id INT PRIMARY KEY AUTO_INCREMENT,  
    order_id INT,  
    product_id INT,  
    quantity INT,  
    price DOUBLE  
);
```

-- BILL

```
CREATE TABLE bill (  
    bill_id INT PRIMARY KEY AUTO_INCREMENT,  
    user_id INT,  
    total_amount DOUBLE,  
    bill_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

-- ACCOUNTS

```
CREATE TABLE accounts (  
    account_id INT PRIMARY KEY AUTO_INCREMENT,  
    user_id INT,  
    balance DOUBLE  
);
```

-- TRANSACTIONS

```
CREATE TABLE transactions (  
    txn_id INT PRIMARY KEY AUTO_INCREMENT,  
    from_acc INT,  
    to_acc INT,  
    amount DOUBLE,  
    txn_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

-- SAMPLE DATA

```
INSERT INTO users(username,password,mobile) VALUES  
('sowmya','123','9999999999');
```

```
INSERT INTO admin(username,password) VALUES  
('admin','admin');
```

```
INSERT INTO products(name, brand, price, quantity) VALUES
```

```
('Yoga Mat Pro','FitLife',1200,15),
```

```
('Stainless Steel Bottle','Milton',600,25),
```

```
('LED Study Lamp','Philips',1500,12),
```

```
('Ergonomic Office Chair','GreenSoul',8500,5),
```

```
('Wireless Phone Stand','Portronics',900,18),
```

```
('Digital Alarm Clock','Casio',1100,10),
```

```
('Mini Air Cooler','Bajaj',4500,6),
```

```
('Electric Kettle','Prestige',1800,14),
```

```
('Vacuum Cleaner','Eureka Forbes',7000,7),
```

```
('Induction Cooktop','Pigeon',2200,11),
```

```
('Smart LED Bulb','Wipro',700,20),
```

```
('Wall Mounted Bookshelf','HomeTown',3500,4),
```



```
('Portable Blender','NutriMix',2500,9),  
('Laptop Backpack','Skybags',2000,13),  
('Noise Cancelling Earplugs','Loop',1500,17),  
('Handheld Garment Steamer','Philips',3200,8),  
('Fitness Resistance Bands','Boldfit',800,22),  
('Tabletop Tripod Stand','Digitek',1200,16),  
('Room Air Purifier','Mi',9000,5),  
('Cordless Screwdriver Kit','Bosch',4000,6);
```

```
SET SQL_SAFE_UPDATES = 0;
```

```
UPDATE products SET brand='Generic', quantity=10 WHERE  
brand IS NULL;
```

```
SELECT * FROM users;
```

```
SELECT * FROM products;
```

```
SELECT * FROM cart;
```

```
SELECT * FROM orders;
```

```
SELECT * FROM transaction_history ;
```

```
SELECT * FROM bill_history ;
```

```
CREATE TABLE bill_history (  
    bill_id INT AUTO_INCREMENT PRIMARY KEY,  
    user_id INT,  
    total_amount DOUBLE,
```

```
bill_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE transaction_history (  
    txn_id INT AUTO_INCREMENT PRIMARY KEY,  
    user_id INT,  
    amount DOUBLE,  
    txn_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

SCREENSHOTS

	8	mango	15	20	3
	9	mixed vege	50	2	3
	10	apple	20	10	4
	11	orange	20	2	4
	12	bannana	12	6	4
•	NULL	NULL	NULL	NULL	NULL

	id	item_name	price	quantity	order_id
▶	1	apple	30	60	1
	2	suger	23	1	1
	3	clips	10	1	1
	4	mixed veg	40	2	1
	5	mango	33	2	2
	6	cucumber	15	1	2
	7	apple	20	2	3
	8	mango	15	20	3
	9	mixed vege	50	2	3
•	NULL	NULL	NULL	NULL	NULL

Result Grid   Filter Rows: Edit:    Export/Import:   Wrap Cell Content: 				
	user_id	username	password	mobile
▶	1	sowmya	123	989999999999
	2	kavith	kavi	NULL
•	NULL	NULL	NULL	NULL

Result Grid

Filter Rows:

Edit:

Export/Import:

	bill_id	user_id	total_amount	bill_date
▶	1	2	22000	2026-05-02 18:44:42
	2	3	140000	2026-05-03 07:53:57
	3	3	4000	2026-05-03 07:57:43
	4	5	2400	2026-05-04 08:32:28
	5	5	4300	2026-05-04 08:52:37
	6	6	2400	2026-05-04 09:22:48
	7	1	14900	2026-05-04 09:42:21
	8	1	4900	2026-05-04 12:11:37
	9	1	1200	2026-05-04 12:56:27
	NULL	NULL	NULL	NULL

bill_history 10 ×

Result Grid

Filter Rows:

Edit:

Export/Import:

	bill_id	user_id	total_amount	bill_date
▶	1	2	22000	2026-05-02 18:44:42
	2	3	140000	2026-05-03 07:53:57
	3	3	4000	2026-05-03 07:57:43
	4	5	2400	2026-05-04 08:32:28
	5	5	4300	2026-05-04 08:52:37
	6	6	2400	2026-05-04 09:22:48
	7	1	14900	2026-05-04 09:42:21
	8	1	4900	2026-05-04 12:11:37
	9	1	1200	2026-05-04 12:56:27
	NULL	NULL	NULL	NULL

bill_history 10

×

99 | bill_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP

Result Grid  Filter Rows: Edit:    Export/Import:  

	product_id	name	price
▶	1	Laptop	50000
	2	Mobile	20000
	3	Headphones	2000
*	NULL	NULL	NULL

90 | bill_id INT AUTO_INCREMENT PRIMARY KEY,

Result Grid  Filter Rows: Edit:    Export/Import:   Wrap Cell Content: 

	order_id	user_id	total_amount	order_date
▶	1	1	152000	2026-05-02 18:05:24
	2	1	120000	2026-05-02 18:08:52
	3	1	0	2026-05-02 18:08:56
	4	2	124000	2026-05-02 18:35:07
	5	2	22000	2026-05-02 18:44:42
*	NULL	NULL	NULL	NULL

90 | SELECT FROM bill_history;

Result Grid  Filter Rows: Edit:    Export/Import:   Wrap Cell Content: 

	txn_id	user_id	amount	txn_date
▶	1	2	22000	2026-05-02 18:44:42
*	NULL	NULL	NULL	NULL

Result Grid

Filter Rows:

Edit:

Export/Import:

Wi

	bill_id	user_id	total_amount	bill_date
▶	1	2	22000	2026-05-02 18:44:42
•	NULL	NULL	NULL	NULL



10.CODING

```
package com.demos;

import java.sql.Connection;
import java.sql.DriverManager;

public class DBConnection {

    public static Connection getConnection() {
        try {
            return DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/shopping_cart",
                "root",
                "root"
            );
        } catch (Exception e) {
            System.out.println(e);
        }
        return null;
    }
}
```

```
}  
}
```

AuthService.java

```
package com.demos;
```

```
import java.sql.*;
```

```
import java.util.Scanner;
```

```
public class AuthService {
```

```
    static Scanner sc = new Scanner(System.in);
```

```
    // REGISTER
```

```
    public static void register() {
```

```
        try {
```

```
            Connection con = DBConnection.getConnection();
```

```
            System.out.print("Enter Username: ");
```

```
            String username = sc.next();
```



```
System.out.print("Enter Password: ");
```

```
String password = sc.next();
```

```
PreparedStatement ps = con.prepareStatement(  
    "INSERT INTO users(username,password) VALUES(?,?)"  
);
```

```
ps.setString(1, username);
```

```
ps.setString(2, password);
```

```
ps.executeUpdate();
```

```
System.out.println("User Registered!");
```

```
} catch (Exception e) {  
    System.out.println(e);  
}
```

```
}
```

```
// LOGIN
```

```
public static int login() {
```

```
    try {
```

```
Connection con = DBConnection.getConnection();

System.out.print("Username: ");
String u = sc.next();

System.out.print("Password: ");
String p = sc.next();

PreparedStatement ps = con.prepareStatement(
    "SELECT * FROM users WHERE username=? AND
password=?"
);

ps.setString(1, u);
ps.setString(2, p);

ResultSet rs = ps.executeQuery();

if (rs.next()) {
    System.out.println("Login Successful!");
    return rs.getInt("user_id");
} else {
    System.out.println("Invalid Login!");
}
```

```
    }

    } catch (Exception e) {
        System.out.println(e);
    }

    return -1;
}
}
```

Shopping Service

```
package com.demos;
```

```
import java.sql.*;
```

```
import java.util.Scanner;
```

```
public class ShoppingService {
```

```
    static Scanner sc = new Scanner(System.in);
```

```
    static void center(String text) {
```

```
int width = 60;

int pad = (width - text.length()) / 2;

System.out.println("\n" + "=".repeat(width));

System.out.println(" ".repeat(Math.max(0, pad)) + text);

System.out.println("=".repeat(width) + "\n");

}
```

```
static void header(String title) {

    System.out.println("\n" + "=".repeat(60));

    center(title);

}
```

```
static void separator() {

    System.out.println("-".repeat(60));

}
```

```
static void footer() {

    System.out.println("\n" + "=".repeat(60));

}
```

```
// ===== MENU =====
```

```
public static void userMenu(int userId) throws Exception {
```

```
Connection con = DBConnection.getConnection();
```

```
while (true) {
```

```
    header("SMARTCART DASHBOARD");
```

```
        System.out.println(" | 1. View Products          | 5. Sort  
Products      |");
```

```
        System.out.println(" | 2. Add to Cart          | 6. Checkout  
|");
```

```
        System.out.println(" | 3. View Cart          | 7. Bill History  
|");
```

```
        System.out.println(" | 4. Search Product      | 8. Transaction  
History |");
```

```
        System.out.println(" |          | 9. Logout          |");
```

```
        separator();
```

```
        System.out.print("Enter your choice (1-9): ");
```

```
        int ch = sc.nextInt();
```

```
        sc.nextLine(); // consume newline
```

```
        switch (ch) {
```

```
            case 1: viewProducts(con); break;
```

```
            case 2: addToCart(con, userId); break;
```

```
            case 3: viewCart(con, userId); break;
```

```

        case 4: searchProduct(con); break;
        case 5: sortProducts(con); break;
        case 6: checkout(con, userId); break;
        case 7: viewBills(con, userId); break;
        case 8: viewTransactions(con, userId); break;
        case 9:
            header("GOODBYE!");
            return;
        default:
            System.out.println("\n❌ Invalid choice! Please try
again.\n");
    }

    System.out.print("\nPress Enter to continue...");
    sc.nextLine();
}
}

```

// ===== PRODUCTS =====

```

static void viewProducts(Connection con) throws
Exception {
    header("ALL PRODUCTS");

```

```
ResultSet rs =  
con.createStatement().executeQuery("SELECT * FROM  
products");
```

```
System.out.printf(" | %-4s | %-25s | %-10s | %-8s  
| \n", "ID", "PRODUCT NAME", "PRICE", "STOCK");
```

```
separator();
```

```
while (rs.next()) {  
    System.out.printf(" | %-4d | %-25s | ₹%9.2f | %7d  
| \n",
```

```
        rs.getInt("product_id"),
```

```
        rs.getString("name"),
```

```
        rs.getDouble("price"),
```

```
        rs.getInt("quantity"));
```

```
    }
```

```
    footer();
```

```
}
```

```
// ===== ADD CART =====
```

```
static void addToCart(Connection con, int userId)
throws Exception {
```

```
    header("ADD TO CART");
```

```
    System.out.print("Enter Product ID: ");
```

```
    int pid = sc.nextInt();
```

```
        PreparedStatement ps =
con.prepareStatement("SELECT quantity FROM products
WHERE product_id=?");
```

```
        ps.setInt(1, pid);
```

```
        ResultSet rs = ps.executeQuery();
```

```
        if (!rs.next()) {
```

```
            System.out.println("\n✗ Product ID " + pid + " not
found!\n");
```

```
            return;
```

```
        }
```

```
        int stock = rs.getInt(1);
```

```
        System.out.print("Enter Quantity (Available: " +
stock + "): ");
```



```
int qty = sc.nextInt();
```

```
if (qty <= 0) {
```

```
    System.out.println("\n✗ Quantity must be  
greater than 0!\n");
```

```
    return;
```

```
}
```

```
if (qty > stock) {
```

```
    System.out.println("\n✗ Not enough stock!  
Only " + stock + " available.\n");
```

```
    return;
```

```
}
```

```
PreparedStatement insert =  
con.prepareStatement(
```

```
    "INSERT INTO cart(user_id,product_id,quantity)  
VALUES(?,?,?) " +
```

```
    "ON DUPLICATE KEY UPDATE quantity = quantity + ?"
```

```
);
```

```
insert.setInt(1, userId);
```

```
insert.setInt(2, pid);  
insert.setInt(3, qty);  
insert.setInt(4, qty);  
insert.executeUpdate();
```

```
System.out.println("\n✅ " + qty + " item(s) added to cart  
successfully!\n");  
}
```

```
// ===== VIEW CART - PERFECTLY ORGANIZED =====
```

```
static void viewCart(Connection con, int userId) throws  
Exception {
```

```
    header("YOUR SHOPPING CART");
```

```
    PreparedStatement ps = con.prepareStatement(  
        "SELECT p.product_id, p.name, p.brand, p.price, c.quantity "  
        +  
        "FROM cart c JOIN products p ON c.product_id=p.product_id  
        WHERE c.user_id=?"  
    );
```

```
    ps.setInt(1, userId);
```

```
    ResultSet rs = ps.executeQuery();
```

```
    double total = 0;
```

```
boolean empty = true;
```

```
System.out.printf(" | %-4s | %-20s | %-6s | %-6s | %-10s  
| \n", "ID", "PRODUCT", "BRAND", "QTY", "SUBTOTAL");  
  
separator();
```

```
while (rs.next()) {  
    empty = false;  
    int pid = rs.getInt(1);  
    String name = rs.getString(2);  
    String brand = rs.getString(3);  
    double price = rs.getDouble(4);  
    int qty = rs.getInt(5);  
    double sub = price * qty;  
    total += sub;
```

```
    System.out.printf(" | %-4d | %-20s | %-6s | %5d | ₹%9.2f  
| \n", pid, name, brand, qty, sub);  
}
```

```
if (empty) {  
    System.out.println(" | YOUR CART IS EMPTY  
| ");
```

```

    } else {
        separator();

        System.out.printf(" |
TOTAL: ₹%9.2f    | \n", total);

    }

    footer();
}

// ===== SEARCH =====

static void searchProduct(Connection con)
throws Exception {

    header("PRODUCT SEARCH");

    System.out.print("Enter keyword to search: ");

    String key = sc.nextLine();

    PreparedStatement ps = con.prepareStatement(

        "SELECT product_id, name, price FROM products WHERE
LOWER(name) LIKE ?"

    );

    ps.setString(1, "%" + key.toLowerCase() + "%");

    ResultSet rs = ps.executeQuery();

```

```
        System.out.printf(" | %-4s | %-30s | %-10s | \n", "ID",  
"PRODUCT NAME", "PRICE");
```

```
        separator();
```

```
        boolean found = false;
```

```
        while (rs.next()) {
```

```
            found = true;
```

```
            System.out.printf(" | %-4d | %-30s | ₹%9.2f | \n",
```

```
                rs.getInt("product_id"),
```

```
                rs.getString("name"),
```

```
                rs.getDouble("price"));
```

```
        }
```

```
        if (!found) {
```

```
            System.out.println(" | NO PRODUCTS FOUND!  
| ");
```

```
        }
```

```
        footer();
```

```
    }
```

```
    // ===== SORT =====
```

```
    static void sortProducts(Connection con) throws Exception {
```

```
header("PRODUCTS - SORTED BY PRICE (LOW TO HIGH)");
```

```
ResultSet rs = con.createStatement().executeQuery(  
    "SELECT product_id, name, price FROM products ORDER BY  
price ASC"  
);
```

```
System.out.printf("| %-4s | %-25s | %-10s |\n", "ID",  
"PRODUCT", "PRICE");
```

```
separator();
```

```
while (rs.next()) {
```

```
    System.out.printf("| %-4d | %-25s | ₹%9.2f |\n",
```

```
        rs.getInt("product_id"),
```

```
        rs.getString("name"),
```

```
        rs.getDouble("price"));
```

```
}
```

```
footer();
```

```
}
```

```
// ===== CHECKOUT - PERFECTLY ORGANIZED =====
```

```
static void checkout(Connection con, int userId) throws  
Exception {
```

```
header("CHECKOUT - REVIEW YOUR CART");
```

```
PreparedStatement ps = con.prepareStatement(  
    "SELECT p.product_id, p.name, p.brand, p.price, c.quantity "  
+  
    "FROM cart c JOIN products p ON c.product_id=p.product_id  
WHERE c.user_id=?"  
);
```

```
ps.setInt(1, userId);
```

```
ResultSet rs = ps.executeQuery();
```

```
double total = 0;
```

```
boolean empty = true;
```

```
System.out.printf(" | %-4s | %-20s | %-6s | %-6s |  
%-10s | \n", "ID", "PRODUCT", "BRAND", "QTY",  
"SUBTOTAL");
```

```
separator();
```

```
while (rs.next()) {
```

```
    empty = false;
```

```
    int pid = rs.getInt(1);
```

```
    String name = rs.getString(2);
```

```
String brand = rs.getString(3);  
double price = rs.getDouble(4);  
int qty = rs.getInt(5);  
double sub = price * qty;  
total += sub;
```

```
System.out.printf(" | %-4d | %-20s | %-6s | %5d |  
₹%9.2f | \n", pid, name, brand, qty, sub);  
}
```

```
if (empty) {  
    System.out.println(" | YOUR CART IS EMPTY  
|");  
    footer();  
    return;  
}
```

```
separator();  
System.out.printf(" | TOTAL:  
₹%9.2f | \n", total);  
footer();
```



```
// Confirmation

System.out.print("\n  Proceed with checkout?
(y/n): ");

String confirm = sc.nextLine().trim().toLowerCase();

if (!confirm.equals("y")) {

    System.out.println("\n  Checkout cancelled!\n");

    return;

}
```

```
// Generate Invoice
```

```
header("INVOICE");
```

```
// Create bill
```

```
PreparedStatement billStmt = con.prepareStatement(

    "INSERT                                INTO
bill_history(user_id,total_amount,bill_date)
VALUES(?,?,NOW())",

    Statement.RETURN_GENERATED_KEYS

);

billStmt.setInt(1, userId);
```

```
billStmt.setDouble(2, total);
```

```
billStmt.executeUpdate();
```

```
ResultSet billRs = billStmt.getGeneratedKeys();
```

```
billRs.next();
```

```
int billId = billRs.getInt(1);
```

```
System.out.println("Invoice #: " + billId);
```

```
System.out.println("Customer: " + userId);
```

```
System.out.println("Date:      "      +      new  
java.sql.Date(System.currentTimeMillis()));
```

```
separator();
```

```
System.out.printf(" | %-4s | %-20s | %-6s |  
%-6s | %-10s | \n", "ID", "PRODUCT", "BRAND",  
"QTY", "AMOUNT");
```

```
separator();
```

```
// Re-query for invoice
```

```
ps = con.prepareStatement(
```

```
    "SELECT  p.product_id,  p.name,  p.brand,  
p.price, c.quantity " +
```

```
        "FROM cart c JOIN products p ON  
c.product_id=p.product_id WHERE c.user_id=?"
```

```
    );
```

```
    ps.setInt(1, userId);
```

```
    rs = ps.executeQuery();
```

```
    while (rs.next()) {
```

```
        int pid = rs.getInt(1);
```

```
        String name = rs.getString(2);
```

```
        String brand = rs.getString(3);
```

```
        double price = rs.getDouble(4);
```

```
        int qty = rs.getInt(5);
```

```
        // Update stock
```

```
        PreparedStatement updateStock =  
con.prepareStatement(
```

```
            "UPDATE products SET quantity=quantity-  
? WHERE product_id=?"
```

```
    );
```

```
    updateStock.setInt(1, qty);
```

```
updateStock.setInt(2, pid);  
updateStock.executeUpdate();
```

```
double sub = price * qty;  
System.out.printf(" | %-4d | %-20s | %-6s |  
%5d | ₹%9.2f | \n", pid, name, brand, qty, sub);  
}
```

```
// Record transaction
```

```
PreparedStatement transStmt =  
con.prepareStatement(  
    "INSERT INTO  
transaction_history(user_id,amount,date)  
VALUES(?,?,NOW())"  
);  
transStmt.setInt(1, userId);  
transStmt.setDouble(2, total);  
transStmt.executeUpdate();
```

```
// Clear cart
```

```
        PreparedStatement clearCart =  
con.prepareStatement("DELETE FROM cart WHERE  
user_id=?");
```

```
        clearCart.setInt(1, userId);
```

```
        clearCart.executeUpdate();
```

```
        separator();
```

```
        System.out.printf(" | GRAND  
TOTAL: ₹%12.2f | \n", total);
```

```
        System.out.println(" | Status:  
 COMPLETED |");
```

```
        footer();
```

```
        center("THANK YOU FOR SHOPPING WITH  
SMARTCART!");
```

```
    }
```

```
// ===== BILL HISTORY =====
```

```
    static void viewBills(Connection con, int userId)  
throws Exception {
```

```
header("BILL HISTORY");
```

```
PreparedStatement ps =  
con.prepareStatement("SELECT * FROM bill_history  
WHERE user_id=? ORDER BY bill_date DESC");
```

```
ps.setInt(1, userId);
```

```
ResultSet rs = ps.executeQuery();
```

```
System.out.printf(" | %-8s | %-12s | %-20s  
|\n", "BILL ID", "AMOUNT", "DATE");
```

```
separator();
```

```
boolean empty = true;
```

```
while (rs.next()) {
```

```
    empty = false;
```

```
    System.out.printf(" | %-8d | ₹%11.2f | %-  
20s |\n",
```

```
        rs.getInt("bill_id"),
```

```
        rs.getDouble("total_amount"),
```

```
        rs.getTimestamp("bill_date"));
```

```
}
```

```
if (empty) {
```

```
    System.out.println(" | NO BILLS  
FOUND! |");
```

```
}
```

```
    footer();
```

```
}
```

```
// ===== TRANSACTION =====
```

```
static void viewTransactions(Connection con, int  
userId) throws Exception {
```

```
    header("TRANSACTION HISTORY");
```

```
        PreparedStatement ps =  
con.prepareStatement("SELECT * FROM  
transaction_history WHERE user_id=? ORDER BY  
date DESC");
```

```
        ps.setInt(1, userId);
```

```
        ResultSet rs = ps.executeQuery();
```

```
        System.out.printf(" | %-12s | %-20s | \n",  
"AMOUNT", "DATE");
```

```
        separator();
```

```
        boolean empty = true;
```

```
        while (rs.next()) {
```

```
            empty = false;
```

```
            System.out.printf(" | ₹%11.2f | %-20s | \n",
```

```
                rs.getDouble("amount"),
```

```
                rs.getTimestamp("date"));
```

```
        }
```

```
        if (empty) {
```

```
            System.out.println(" | NO  
TRANSACTIONS! | ");
```

```
        }
```

```
        footer();
```

```
    }
```



```
}
```

MainApp.java

```
package com.demos;
```

```
import java.util.Scanner;
```

```
public class MainApp {
```

```
    static Scanner sc = new Scanner(System.in);
```

```
    public static void main(String[] args) throws Exception {
```

```
        while (true) {
```

```
            System.out.println("\n1. Register");
```

```
            System.out.println("2. Login");
```

```
            System.out.println("3. Exit");
```

```
int ch = sc.nextInt();
```

```
switch (ch) {
```

```
    case 1:
```

```
        AuthService.register();
```

```
        break;
```

```
    case 2:
```

```
        int userId = AuthService.login();
```

```
        if (userId != -1)
```

```
            ShoppingService.userMenu(userId);
```

```
        break;
```

```
    case 3:
```

```
        System.exit(0);
```

```
    }
```

```
}
```

```
}
```

```
}
```

OUTPUTS

```
MainApp [Java Application] C:\Users\Sowmya\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.f
=====
WELCOME TO SMARTCART PRO
=====
1. Register New Account
2. Login to Account
3. Exit Application
=====
Enter your choice (1-3): |
```

```
=====
NEW USER REGISTRATION
=====
Enter Username: hari prasad
Enter Password: 12345
-----
[X] Registration Successful!
    Username: hari prasad
    User ID: 9
    You can now login with these credentials!
-----
Press Enter to continue...
```

☒ Login Successful! Welcome back!

User ID: 9

Username: hari prasad

=====

SMARTCART DASHBOARD

=====

- | | |
|-------------------|------------------------|
| 1. View Products | 5. Sort Products |
| 2. Add to Cart | 6. Checkout |
| 3. View Cart | 7. Bill History |
| 4. Search Product | 8. Transaction History |
| | 9. Logout |
-

Enter your choice (1-9):



ALL PRODUCTS			
ID	PRODUCT NAME	PRICE	STOCK
1	Yoga Mat Pro	₹ 1200.00	12
2	Steel Bottle	₹ 600.00	17
3	LED Study Lamp	₹ 1500.00	8
4	Office Chair	₹ 8500.00	4
5	Portable Blender	₹ 2500.00	9
6	Fitness Bands	₹ 800.00	16
7	Electric Kettle	₹ 1800.00	14
8	Vacuum Cleaner	₹ 7000.00	7
9	Induction Stove	₹ 2200.00	11
10	Smart Bulb	₹ 700.00	20
11	Bookshelf	₹ 3500.00	4
12	Laptop Bag	₹ 2000.00	13
13	Air Purifier	₹ 9000.00	5
14	Tripod Stand	₹ 1200.00	16
15	Power Bank	₹ 2200.00	15
16	Wireless Charger	₹ 1800.00	10
17	Gaming Pad	₹ 1500.00	12
18	Mini Projector	₹ 6000.00	6
19	Desk Organizer	₹ 900.00	18
20	Water Filter	₹ 7500.00	7
Press Enter to continue...			

YOUR SHOPPING CART				
ID	PRODUCT	BRAND	QTY	SUBTOTAL
1	Yoga Mat Pro	FitLife	1	₹ 1200.00
4	Office Chair	GreenSoul	2	₹ 17000.00
8	Vacuum Cleaner	Eureka	5	₹ 35000.00
				TOTAL: ₹ 53200.00
Press Enter to continue...				

PRODUCTS - SORTED BY PRICE (LOW TO HIGH)			
ID	PRODUCT	PRICE	
2	Steel Bottle	₹ 600.00	
10	Smart Bulb	₹ 700.00	
6	Fitness Bands	₹ 800.00	
19	Desk Organizer	₹ 900.00	
1	Yoga Mat Pro	₹ 1200.00	
14	Tripod Stand	₹ 1200.00	
3	LED Study Lamp	₹ 1500.00	
17	Gaming Pad	₹ 1500.00	
7	Electric Kettle	₹ 1800.00	
16	Wireless Charger	₹ 1800.00	
12	Laptop Bag	₹ 2000.00	
9	Induction Stove	₹ 2200.00	
15	Power Bank	₹ 2200.00	
5	Portable Blender	₹ 2500.00	
11	Bookshelf	₹ 3500.00	
18	Mini Projector	₹ 6000.00	
8	Vacuum Cleaner	₹ 7000.00	
20	Water Filter	₹ 7500.00	
4	Office Chair	₹ 8500.00	
13	Air Purifier	₹ 9000.00	

```
=====
SMARTCART DASHBOARD
=====
```

- | | |
|-------------------|------------------------|
| 1. View Products | 5. Sort Products |
| 2. Add to Cart | 6. Checkout |
| 3. View Cart | 7. Bill History |
| 4. Search Product | 8. Transaction History |
| | 9. Logout |

```
-----
Enter your choice (1-9): 6
=====
```

```
=====
FINAL BILL
=====
```

```
Air Purifier x1 = ₹9000.0
Portable Blender x3 = ₹7500.0
```

```
TOTAL: ₹16500.0
=====
```

```
Press Enter to continue...
=====
```

=====

BILL HISTORY

=====

Bill ID: 15 ₹16500.0 2026-05-04 14:39:24.0
Bill ID: 6 ₹2400.0 2026-05-04 09:22:48.0

=====

Press Enter to continue...

=====

=====

SMARTCART DASHBOARD

=====


```
=====
                                TRANSACTION HISTORY
                                =====
```

```
₹16500.0  2026-05-04 14:39:24.0
₹2400.0   2026-05-04 09:22:48.0

=====
```

```
Press Enter to continue...

=====
```

```
=====
                                SMARTCART DASHBOARD
                                =====
```

- | | |
|-------------------|------------------------|
| 1. View Products | 5. Sort Products |
| 2. Add to Cart | 6. Checkout |
| 3. View Cart | 7. Bill History |
| 4. Search Product | 8. Transaction History |
| | 9. Logout |

```
-----
Enter your choice (1-9): 9

=====
```

```
=====
                                GOODBYE!
                                =====
```

```
Press Enter to continue...

=====
```

FULL WORKING FLOW (END TO END)

► 1. PROGRAM STARTS

👉 You run:

MainApp.java

Execution starts here:

```
public static void main(String[] args)
```

👉 It shows:

- | | |
|----|----------|
| 1. | Register |
| 2. | Login |
| 3. | Exit |
-

👤 2. REGISTER FLOW

👉 If user enters:

1

Method called:

```
AuthService.register();
```

What happens:

1. Takes input:

- username
- password

2. Connects to DB

3. Executes:

```
INSERT INTO users(username,password)
VALUES(?,?)
```

✓ User stored in database

3. LOGIN FLOW

 If user enters:

2

Method called:

```
AuthService.login();
```

What happens:

1. Takes username & password

2. Executes:

```
SELECT * FROM users WHERE username=? AND  
password=?
```

✓ If correct:

- Returns user_id
- Moves to shopping menu:

```
ShoppingService.userMenu(userId);
```

✗ If wrong:

- Prints:

Invalid Login!

4. SHOPPING MENU FLOW

After login:

1. View Products
2. Add Item
3. View Cart
4. Remove Item
5. Checkout
6. View Bills

7. View Transactions

8. Logout

5. VIEW PRODUCTS

 Calls:

`viewProducts()`

Query:

```
SELECT * FROM products;
```

✓ Shows all items

6. ADD ITEM TO CART

 Calls:

`addToCart(userId)`

Steps:

1. User enters:

- `product_id`

- quantity

2. Executes:

```
INSERT INTO cart(user_id,product_id,quantity)
```

✓ Item stored in cart table

👁️ 7. VIEW CART

👉 Calls:

```
viewCart(userId)
```

Query:

```
SELECT p.name, p.price, c.quantity
```

```
FROM cart c
```

```
JOIN products p ON c.product_id = p.product_id
```

```
WHERE user_id = ?
```

✓ Shows:

- product name
- price
- quantity
- subtotal
- total

✖ 8. REMOVE ITEM

👉 Calls:

`removeItem(userId)`

Query:

`DELETE FROM cart WHERE user_id=? AND
product_id=?`

✓ Item removed

💳 9. CHECKOUT (MOST IMPORTANT)

👉 Calls:

`checkout(userId)`

12
34 Step-by-step inside checkout:

1 Get cart data

`SELECT price, quantity FROM cart`

👉 Calculate:

$\text{total} = \text{price} \times \text{quantity}$

2 Insert into ORDERS

INSERT INTO orders(user_id,total_amount)

✓ Stores current order

3 Insert into BILL HISTORY

INSERT INTO bill_history(user_id,total_amount)

✓ Stores permanent bill

4 Insert into TRANSACTION HISTORY

INSERT INTO transaction_history(user_id,amount)

✓ Stores payment record

5 Clear cart

DELETE FROM cart WHERE user_id=?

✓ Cart becomes empty

✓ Output:

Order placed!
Total: XXXX

10. VIEW BILL HISTORY

👉 Calls:

viewBills(userId)

Query:

```
SELECT * FROM bill_history WHERE user_id=?
```

✓ Shows:

- bill id
 - amount
 - date
-

VIEW TRANSACTION HISTORY

👉 Calls:

viewTransactions(userId)

Query:

```
SELECT * FROM transaction_history WHERE  
user_id=?
```

✓ Shows:

- transaction id
 - amount
 - date
-

LOGOUT

👉 Returns to main menu

START



Main Menu



Register / Login



Shopping Menu



Cart → Checkout



Orders + Bill + Transaction Stored



View History



END

SYSTEM	IMPLEMENTATION	(CODE
--------	----------------	-------

EXPLANATION)

Database Connection

- Uses JDBC Driver
 - Connects using URL, username, and password
-

Add Item

- Inserts item into items table
 - order_id is NULL → means item is in cart
-

Remove Item

- Deletes item where order_id IS NULL
 - Only cart items can be removed
-

View Cart

- Displays items where order_id IS NULL
-

Generate Bill

- Fetches all cart items
- Calculates total amount
- Creates a new record in orders table

- Assigns order_id to all cart items

WORKING LOGIC

- Items added → stored with order_id = NULL
- When bill generated:
 1. Total calculated
 2. New order created
 3. Items updated with order_id
- Items become part of that order
- The system successfully integrates Java and MySQL using JDBC to perform all operations like user management, cart handling, billing, and transaction tracking efficiently.

CONCLUSION

The Shopping Cart System successfully demonstrates the implementation of a real-world application using Core Java, JDBC, and MySQL. The system allows users to efficiently manage cart items, perform database operations, and generate bills automatically.

By introducing the Orders and Items tables with a one-to-many relationship, the project improves data organization and enables proper order management. It also ensures data persistence, reduces manual errors, and provides a structured approach to handling shopping transactions.

Overall, the project enhances understanding of database connectivity, SQL operations, and application logic in Java.

FUTURE ENHANCEMENTS

The system can be further improved by implementing the following features:

- Development of a Graphical User Interface (GUI) using Java Swing or JavaFX
- Implementation of user authentication (login and registration system)
- Addition of order history and previous purchase tracking
- Integration of online payment methods
- Conversion of the system into a web-based application
- Addition of product categories and search functionality